

DATA-260 Homework 1 Report

Section 0 — Personal Configuration

SID4	6758
PORT_BASE	8458
PREFIX	s6758
SEED	6758
VERIFY_SEED	266758
DOMAIN_ID	6 — Rental housing listings

1. Project Overview

This project implements a rental housing listings web application (DOMAIN_ID 6) built with HTML, CSS, and vanilla JavaScript, covering form validation, JSON serialization, object destructuring, the spread operator, and a closure-based submission counter. The application was containerized with Docker and Nginx, tested locally, and deployed to AWS ECS Fargate behind a public IP. A second component implements a three-agent (Planner → Reviewer → Finalizer) pipeline using LangChain and a local Ollama model to generate topical tags and a summary from the listing content. The pipeline's non-determinism was measured across 40 runs at two temperature settings, and a reusable model-adapter (Part 4) tracks token usage and conversation statistics across a multi-turn conversation.

2. Docker and Local Deployment

The application (index.html, script.js) was containerized using an nginx:alpine base image and built/run locally:

- `docker build -t s6758-web-app .`
- `docker run -d -p 8458:80 --name s6758-web-app-container s6758-web-app`

The container ran successfully and the form was tested end-to-end on localhost, confirming JSON.stringify/parse, destructuring, the spread operator, and the closure-based submission counter all fire correctly on submit.

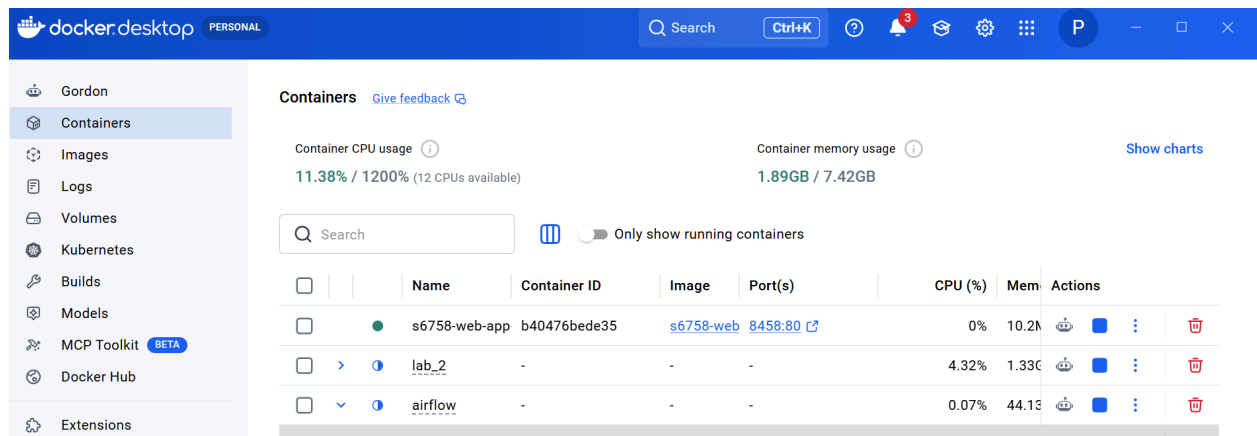


Figure 1. Docker Desktop showing the s6758-web-app container running and mapped to port 8458.

The screenshot shows a web browser at localhost:8458 displaying a 'Rental Housing Listings' form. The form fields are as follows:

- Property Address ***: 385 River Oaks Parkway
- Monthly Rent ***: 3500
- Your Email ***: pragna.chourasia@sjsu.edu
- Property Description ***: Decent property, good community
- Category ***: Apartment
- ☒ I agree to the terms and conditions.
- Submit Listing** button

Figure 2. Rental Housing Listings form running at localhost:8458.

The screenshot shows the same web browser at localhost:8458, but now displaying the 'Submitted Listings' section. The form fields are partially filled:

- you@sjsu.edu** (in the email field)
- Property Description ***: Describe the property: bedrooms, bathrooms, square footage, amenities, pet policy, etc.
- Category ***: Choose
- ☐ I agree to the terms and conditions.
- Submit Listing** button

Below the form, the 'Submitted Listings:' section shows a list of submitted listings:

- 385 River Oaks Parkway | Apartment | 3500
- Delete** button

Figure 3. Submitted listing rendered in the UI after a successful form submission on localhost.

3. AWS Deployment

The image was pushed to Amazon ECR and deployed as a single-task AWS ECS Fargate service.

ECR repository	my-web-app (tag: latest)
ECS cluster	my-web-app-cluster
ECS service	my-web-app-service
Public URL	http://13.56.18.151

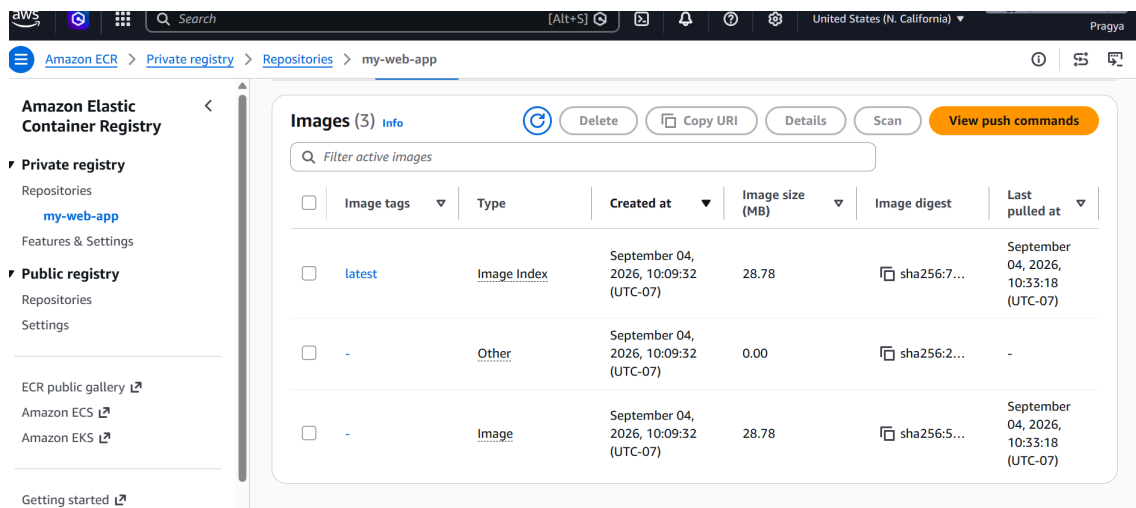


Figure 4. ECR repository my-web-app showing the pushed image tagged latest.

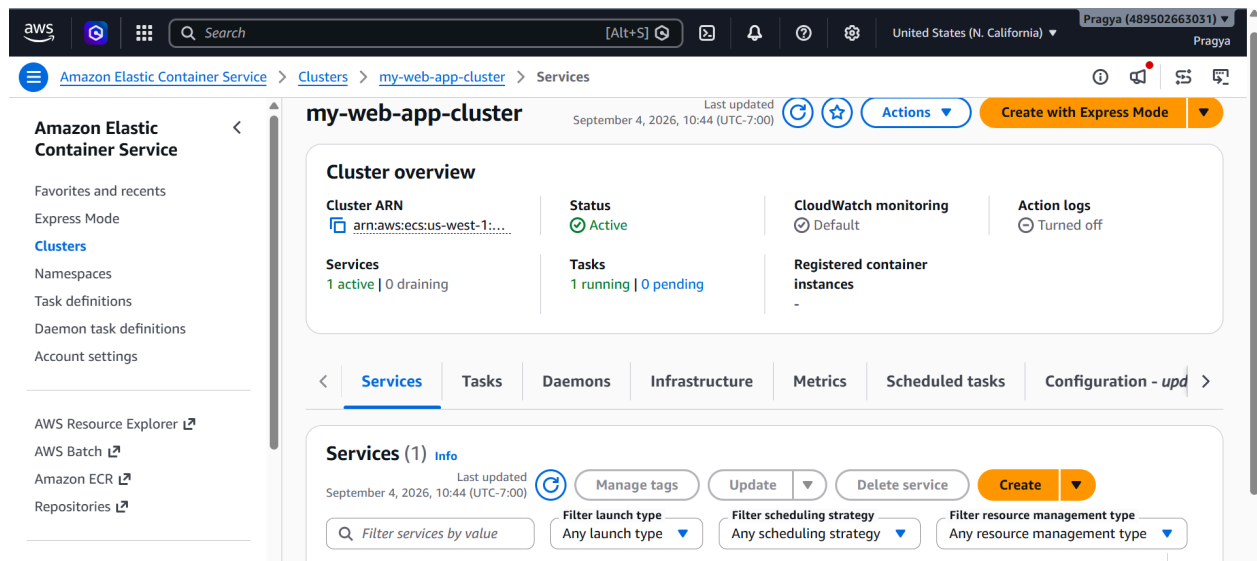


Figure 5. ECS cluster my-web-app-cluster showing 1 active service and 1 running task.

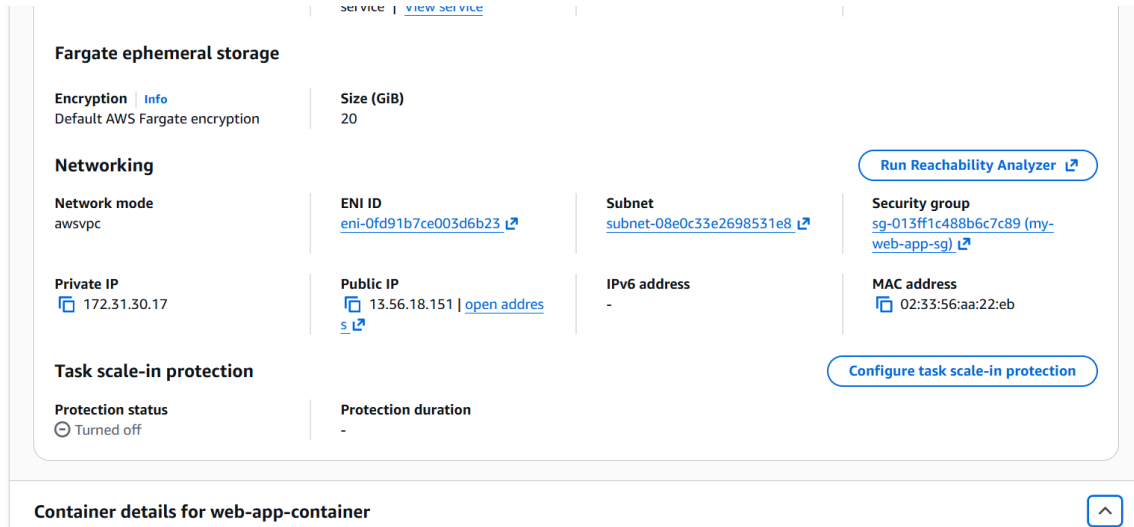


Figure 6. Running ECS task's networking details, showing assigned public IP 13.56.18.151.

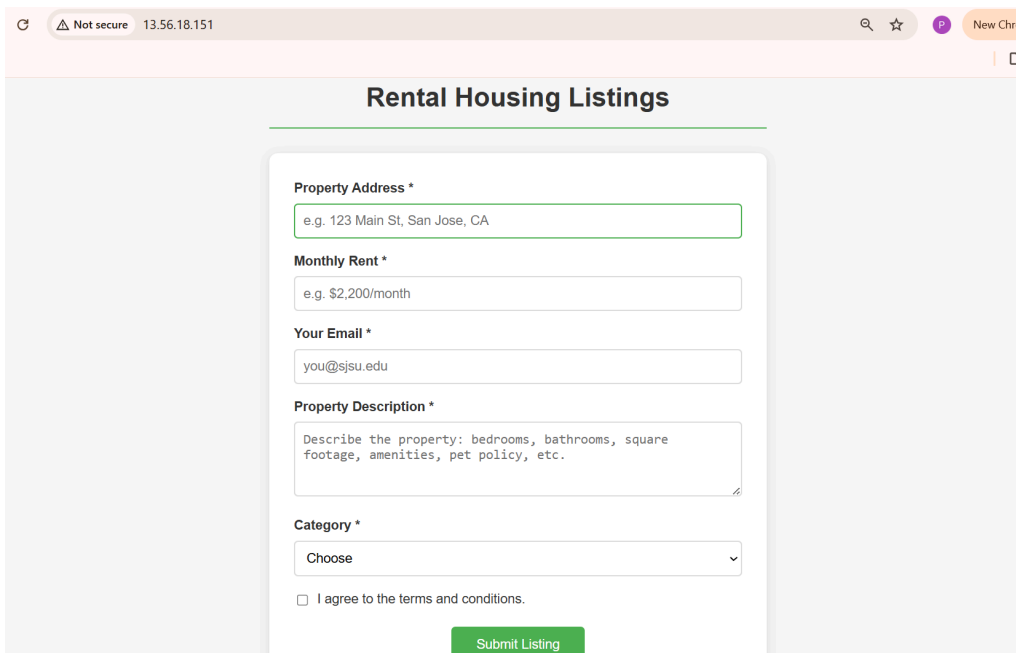


Figure 7. Rental Housing Listings form loaded from the public IP, confirming the ECS deployment is reachable externally.

4. Part 2 — Agent Pipeline Results

Command used: `python agents_demo.py`

Q1. Final tags obtained

Downtown San Jose, Updated Kitchen, In-Unit Laundry

Q2. Final summary

Bright 1400 sq ft 2BR apartment near downtown San Jose offers updated kitchen, in-unit laundry, and covered parking.

Q3. Did the Reviewer agent change anything?

Yes. The Reviewer trimmed two of the Planner's three tags ("Near Downtown San Jose" → "Downtown San Jose"; "Updated Kitchen Appliances" → "Updated Kitchen") and reworded the summary from "Spacious 2BR apartment..." to "Bright 2BR apartment...". The third tag, "In-Unit Laundry", was left unchanged. This shows the Reviewer step performs real validation/tightening rather than echoing the Planner's output verbatim.

Pipeline explanation

The Planner agent takes the rental listing's title and description and proposes 3 topical tags plus a one-sentence summary. The Reviewer agent receives the Planner's output as conversation history and checks whether the tags are specific and the summary fits the word limit, tightening anything too generic or too long. The Finalizer agent uses the Reviewer's feedback to produce the locked-in tags and summary, with issues cleared to an empty list. Each step's raw model reply is parsed as JSON and normalized into a fixed schema (`coerce_reply()`), so a slightly malformed model response still produces valid, usable output.

```
(venv) PS C:\SJSU Assignments\Agentic AI\Assignment_1> python agents_demo.py

--- Planner (180620 ms) ---
{
  "thought": "Identified key features like location, amenities, and pet policies to create relevant tags and a concise s
ummary.",
  "message": "Tags focus on location, kitchen updates, and laundry. Summary highlights size, proximity, and key perks.",
  "data": {
    "tags": [
      "Near Downtown San Jose",
      "Updated Kitchen Appliances",
      "In-Unit Laundry"
    ],
    "summary": "Spacious 2BR apartment near downtown San Jose with updated kitchen, in-unit laundry, and parking.",
    "issues": []
  }
}
```

Figure 8. Planner agent output for the fixed rental listing input.

```

Windows PowerShell
--- Reviewer (37769260 ms) ---
{
  "thought": "Tags focus on location, kitchen updates, and laundry. Summary highlights size, proximity, and key perks.",
  "message": "Tags are specific and relevant. Summary concisely captures key details without exceeding word limit.",
  "data": {
    "tags": [
      "Downtown San Jose",
      "Updated Kitchen",
      "In-Unit Laundry"
    ],
    "summary": "Bright 2BR apartment near downtown San Jose with updated kitchen, laundry, and parking.",
    "issues": []
  }
}

Finalized Output
{
  "thought": "Finalizing tags based on location, kitchen updates, and laundry as per feedback. Summary emphasizes size, proximity, and key amenities.",
  "message": "Tags and summary align with feedback: specific, relevant, and concise. No issues detected.",
  "data": {
    "tags": [
      "Downtown San Jose",
      "Updated Kitchen",
      "In-Unit Laundry"
    ],
    "summary": "Bright 1400 sq ft 2BR apartment near downtown San Jose offers updated kitchen, in-unit laundry, and covered parking.",
    "issues": []
  }
}

```

Figure 9. Reviewer and Finalized Output blocks, showing the tags being tightened.

```

Publish Package
{
  "title": "1400 Sq Ft 2BR Apartment Near Downtown San Jose",
  "email": "pragya.chourasia@sjsu.edu",
  "content": "Bright 2 bedroom, 1 bath apartment on the second floor, updated kitchen with new appliances, in-unit laundry, one covered parking spot, small balcony, walking distance to light rail. No smoking. Cats allowed with deposit.",
  "agents": {
    "transcript": [
      {
        "role": "Planner",
        "content": "Tags focus on location, kitchen updates, and laundry. Summary highlights size, proximity, and key perks."
      },
      {
        "role": "Reviewer",
        "content": "Tags are specific and relevant. Summary concisely captures key details without exceeding word limit."
      }
    ],
    "final": {
      "tags": [
        "Downtown San Jose",
        "Updated Kitchen",
        "In-Unit Laundry"
      ],
      "summary": "Bright 1400 sq ft 2BR apartment near downtown San Jose offers updated kitchen, in-unit laundry, and covered parking.",
      "issues": []
    }
  },
  "submissionDate": "2026-09-04T16:06:45Z"
}

```

Figure 10. Final Publish Package JSON, combining the listing data with the agent transcript and final tags/summary.

5. Part 3 — Measuring Non-Determinism

The pipeline was run 40 times total on one fixed input (saved once to cases/nondeterminism_input.json) — 20 runs at temperature 0.7 and 20 at temperature 0.0 — using llama3.2:3b in place of qwen3:8b to keep the batch runtime practical.

Temperature 0.7 produced 15 distinct tag sets and 17 distinct summaries across 20 runs. No tag appeared in all 20 runs, and 13 tags appeared exactly once. Latency was 25,517 ms at p50, 37,498 ms at p95, and 37,498 ms at p99. Temperature 0.0 produced 2 distinct tag sets and 1 distinct summary

across 20 runs. No exact tag string appeared in all 20 runs because of capitalization/wording differences, and no tag appeared exactly once. Latency was 23,508 ms at p50, 26,898 ms at p95, and 26,898 ms at p99. At temperature 0.0, the dominant tag set appeared 18 of 20 times and the alternate set appeared twice; the summary was identical in all 20 runs.

Metric	Temp 0.7	Temp 0.0
Distinct tag sets	15	2
Tags in all 20 runs	None	None
Tags in exactly 1 run	13: Luxury Housing; Near Downtown; Apartments for Rent; Real Estate; Apartment; Pet Friendly; 2BR Apartment; Rentals; Apartment for rent; Downtown area; Rent; Apartment For Rent; Downtown Area	None
Latency p50 (ms)	25,517	23,508
Latency p95 (ms)	37,498	26,898
Latency p99 (ms)	37,498	26,898

Why temperature 0.7 is expected to vary more than 0.0: temperature controls how much the model's next-token sampling deviates from always picking the single highest-probability token. At temperature 0.0, sampling is effectively greedy/deterministic, so the same prompt tends to produce the same tags and wording run after run (modulo minor system-level non-determinism). At 0.7, the model samples from a wider probability distribution at each token, so small differences early in the response (e.g. word choice or capitalization in a tag) can compound into visibly different final tag sets across runs, even though the underlying rental listing content stays fixed.

6. Part 4 — Token Accounting

A reusable model-adapter module was implemented with a stable complete(messages, tools=None) interface, and a /stats command that reports turn count, cumulative token counts, and serialized conversation-history length. A five-turn conversation was completed. After turn 3, the totals were 736 input tokens, 243 output tokens, and 979 total tokens, with a serialized history length of 1,939 characters. After turn 5, the totals were 1,707 input tokens, 421 output tokens, and 2,128 total tokens, with a serialized history length of 2,988 characters.

Stats after turn 3

Metric	Value
Turn count	3
Cumulative input tokens	736

Cumulative output tokens	243
Cumulative total tokens	979
Serialized history length	1939

```

You: What security issue should I check when accepting user input?
Input tokens: 243
Output tokens: 189
Total tokens: 352

Assistant:
- Input Validation:
  - Always validate user input to prevent SQL injection, cross-site scripting (XSS), and other attacks.
- Sanitization:
  - Sanitize user input to remove any malicious characters or code.
- Escaping:
  - Escape user input to prevent it from being executed as code.
- Data Type Checking:
  - Check the data type of user input to ensure it matches the expected type.
- Authentication:
  - Verify the authenticity of user input to prevent tampering or manipulation.

You: Suggest one test for the add function.
Input tokens: 370
Output tokens: 34
Total tokens: 404

Assistant:
- Test Case:
  - Test the function with negative numbers to ensure it returns the correct result: `add(-1, -2) == -3`

You: /stats

Stats:
{'turn_count': 3, 'cumulative_input_tokens': 736, 'cumulative_output_tokens': 243, 'cumulative_total_tokens': 979, 'serialized_history_length': 1939}

You: |

```

Figure 12. /stats output after turn 3.

Stats after turn 5

Metric	Value
Turn count	5
Cumulative input tokens	1707
Cumulative output tokens	421
Cumulative total tokens	2128
Serialized history length	2988

```

You: Review this code: result = eval(user_input)
Input tokens: 424
Output tokens: 100
Total tokens: 524

Assistant:
- Correctness:
  - The code is correct in terms of its intended purpose, but it poses a significant security risk.
- Bugs:
  - No bugs found.
- Security Concerns:
  - The use of 'eval()' can lead to code injection attacks, allowing an attacker to execute arbitrary code.
- Missing Tests:
  - No tests found.
- Maintainability Issues:
  - The use of 'eval()' makes the code difficult to debug and maintain.

You: Give me the three most important review priorities from this conversation.
Input tokens: 545
Output tokens: 77
Total tokens: 622

Assistant:
- Input Validation:
  - Always validate user input to prevent SQL injection, cross-site scripting (XSS), and other attacks.
- Security Concerns:
  - The use of 'eval()' poses a significant security risk and should be avoided.
- Correctness:
  - Ensure the code is correct and functions as intended, including handling potential errors and edge cases.

You: /stats

Stats:
{'turn_count': 5, 'cumulative_input_tokens': 1707, 'cumulative_output_tokens': 421, 'cumulative_total_tokens': 2128, 'serialized_history_length': 2988}

```

Figure 13. /stats output after turn 5.

Discussion

Prior conversation context is present with every turn because the underlying chat completion API is stateless — the model has no memory between calls, so the full message history must be included each time for the model to have any awareness of earlier turns. A system prompt differs from a user message in that it sets persistent behavior/role instructions for the whole conversation and is typically weighted differently by the model, whereas a user message is one turn of the actual back-and-forth content. Input tokens grow over a conversation because each new turn's request includes the entire prior history, not just the newest message, so cumulative input token count grows roughly linearly (or faster) with turn count. This growth is eventually limited by the model's context window (maximum tokens it can process in one request) — once history plus the new message would exceed that limit, older turns must be truncated, summarized, or dropped.

AI Use Summary

A detailed AI_USE.md is included in the repository. In brief: an AI assistant was used to scaffold the HTML/JS/Docker/agent code, walk through Docker/Ollama/AWS CLI setup, and debug real errors as they occurred. One concrete example of an AI-produced starting point that was wrong: the original agents_demo.py skeleton imported ChatOllama from langchain_community.chat_models, which failed with ImportError on the installed package versions (ChatOllama had moved to the standalone langchain_ollama package). This was caught by actually running the script and reading the traceback, and fixed by changing the import to from langchain_ollama import ChatOllama, after which the pipeline ran successfully.